Deep Reinforcement Learning for Multi-Asset Trading

Rousomanis Georgios (10703)

May 2026

Motivation

- Financial markets are sequential, stochastic, and non-stationary
- Classical strategies struggle under regime shifts
- DRL enables adaptive decision-making directly from data

System Overview

Full Pipeline

- Custom multi-asset trading environments
- Deep Reinforcement Learning agents
- Temporal feature modeling (MLP, LSTM, Transformer)
- Risk control (Fuzzy Logic)
- Hyperparameter optimization (Genetic Algorithm)

Trading Environment

- Multi-asset portfolio (stocks + crypto)
- Gymnasium-based sequential decision process
- Observation contains:
  - Portfolio state (cash + holdings)
  - Market state (prices + indicators)
  - Risk signals (VIX, turbulence for stocks)

Trading Environment

- Multi-asset portfolio (stocks + crypto)
- Gymnasium-based sequential decision process
- Observation contains:
  - Portfolio state (cash + holdings)
  - Market state (prices + indicators)
  - Risk signals (VIX, turbulence for stocks)

State representation:

$$s_t = [s_t^{portfolio}, s_t^{market}]$$

Trading Environment

- Multi-asset portfolio (stocks + crypto)
- Gymnasium-based sequential decision process
- Observation contains:
  - Portfolio state (cash + holdings)
  - Market state (prices + indicators)
  - Risk signals (VIX, turbulence for stocks)

State representation:

$$s_t = [s_t^{portfolio}, s_t^{market}]$$

Continuous action space:

$$a_t \in [-1, 1]^N$$

Trading Environment

- Multi-asset portfolio (stocks + crypto)
- Gymnasium-based sequential decision process
- Observation contains:
  - Portfolio state (cash + holdings)
  - Market state (prices + indicators)
  - Risk signals (VIX, turbulence for stocks)

State representation:

$$s_t = [s_t^{portfolio}, s_t^{market}]$$

Continuous action space:

$$a_t \in [-1, 1]^N$$

Reward:

$$r_t = \log\left(\frac{V_{t+1} + \epsilon}{V_t + \epsilon}\right) - \lambda e_t v_t$$

- e_t: portfolio exposure
- v_t: market volatility
- λ: risk sensitivity coefficient

Policy Architectures

- MLP (baseline)
- LSTM (temporal memory)
- Transformer (attention-based modeling)
- Hybrid MLP–Transformer

Key Challenge

Financial data requires modeling both:

- temporal dependencies
- cross-asset interactions

Transformer Feature Extractor

Input Construction

$$X = [x_{t-T+1}, \ldots, x_t]$$

where each observation contains both portfolio and market information:

$$x_t = [s_t^{portfolio}, s_t^{market}]$$

Architecture

- Sequence length: $T = 10$
- Linear projection to $d_{model} = 128$
- Learnable positional embeddings
- Transformer Encoder:
  - 2 encoder layers
  - 4 attention heads
  - Feed-forward dimension 256
  - Dropout 0.1

Final representation:

$$h_t = Transformer(X)_T$$

The embedding h_t is provided to the actor and critic networks.

Hybrid MLP–Transformer Architecture

Motivation

Market observations exhibit temporal structure, while portfolio information represents an instantaneous allocation state.

Hybrid MLP–Transformer Architecture

Motivation

Market observations exhibit temporal structure, while portfolio information represents an instantaneous allocation state.

Market Branch

$$X^{market} = [s^{market}_{t-T+1}, \ldots, s^{market}_{t}]$$

$$h_t = Transformer(X^{market})_T$$

Hybrid MLP–Transformer Architecture

Motivation

Market observations exhibit temporal structure, while portfolio information represents an instantaneous allocation state.

Market Branch

$$X^{market} = [s^{market}_{t-T+1}, \ldots, s^{market}_{t}]$$

$$h_t = Transformer(X^{market})_T$$

Portfolio Branch

$$h^{portfolio}_t = MLP(s^{portfolio}_t)$$

Fusion

$$z_t = [h^{market}_t, h^{portfolio}_t]$$

Concatenation followed by a projection layer before being passed to the actor and critic networks.

Data Processing

- Rolling window sequences ($T = 10$)
- Online feature-wise normalization
- Reward normalization via rolling statistics

Why it matters

- Handles non-stationary distributions
- Prevents scale dominance

Reinforcement Learning Algorithms

On-Policy Methods

A2C

- Actor-Critic architecture
- Uses advantage estimates to reduce variance
- Stable but less sample efficient

PPO

- Clipped policy updates
- Prevents large policy shifts
- Widely used due to strong stability

Off-Policy Methods

DDPG

- Deterministic actor-critic
- Replay buffer for sample reuse
- Designed for continuous actions

TD3

- Extension of DDPG
- Twin critics reduce overestimation bias
- Delayed policy updates improve stability

SAC

- Stochastic actor-Critic
- Strong sample efficiency

Training Performance

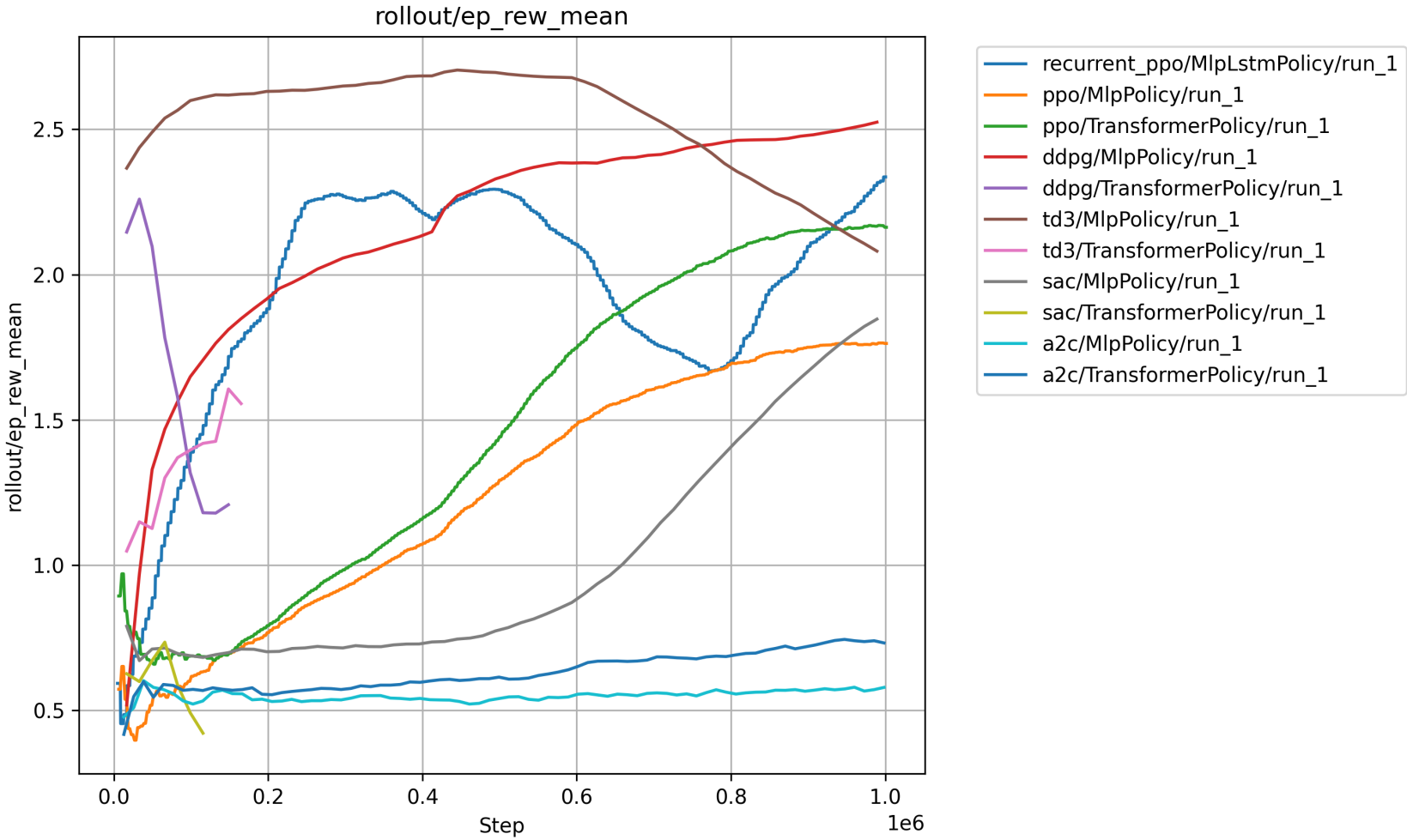


Figure: Training reward evolution

Financial Performance

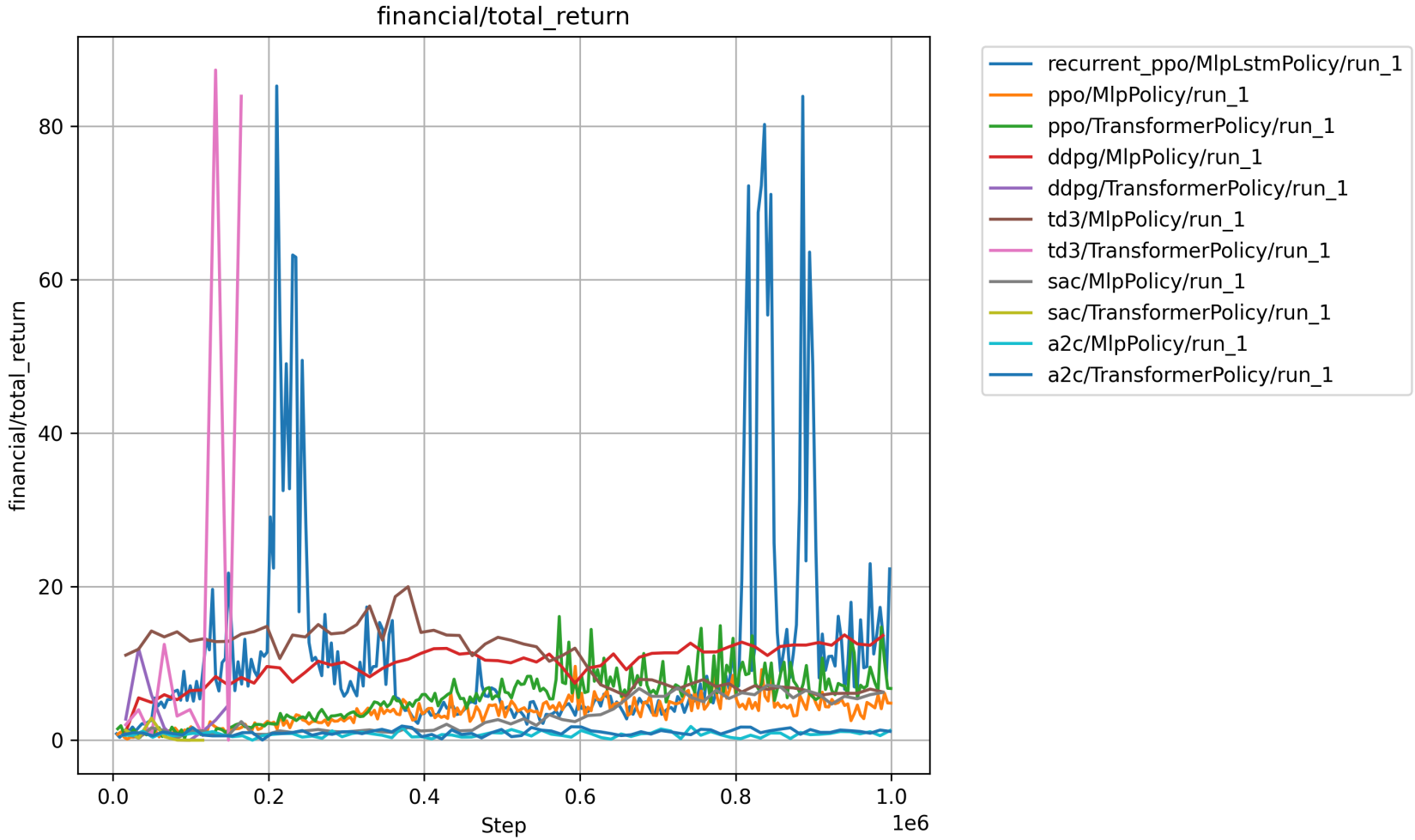


Figure: Total return across models

Risk-Adjusted Performance

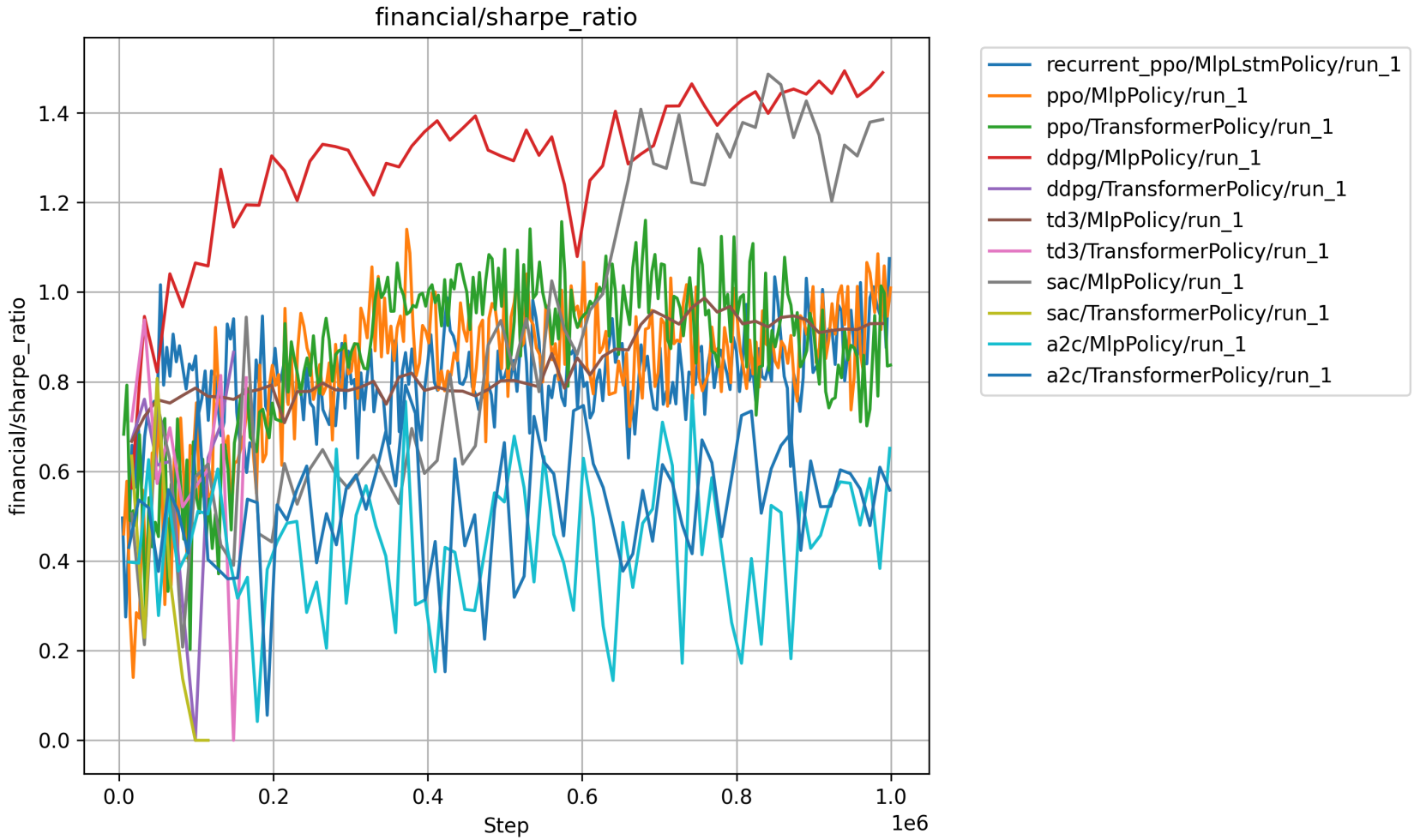


Figure: Sharpe ratio comparison

Key Findings

- PPO $\rightarrow$ most stable learning behavior
- DDPG (MLP) $\rightarrow$ strongest overall performance
- Transformers $\rightarrow$ higher representation power, lower stability (off-policy)
- LSTM $\rightarrow$ strong but unstable long-horizon behavior

Backtesting Evaluation

Evaluation Setup

- Out-of-sample period:
  - Jan 2026 – Mar 2026
- Dow Jones 30 constituents
- Metrics:
  - Total Return
  - Portfolio Value
  - Sharpe Ratio

Selected Models

- DDPG (MLP)
- PPO (Transformer)
- PPO (Hybrid)

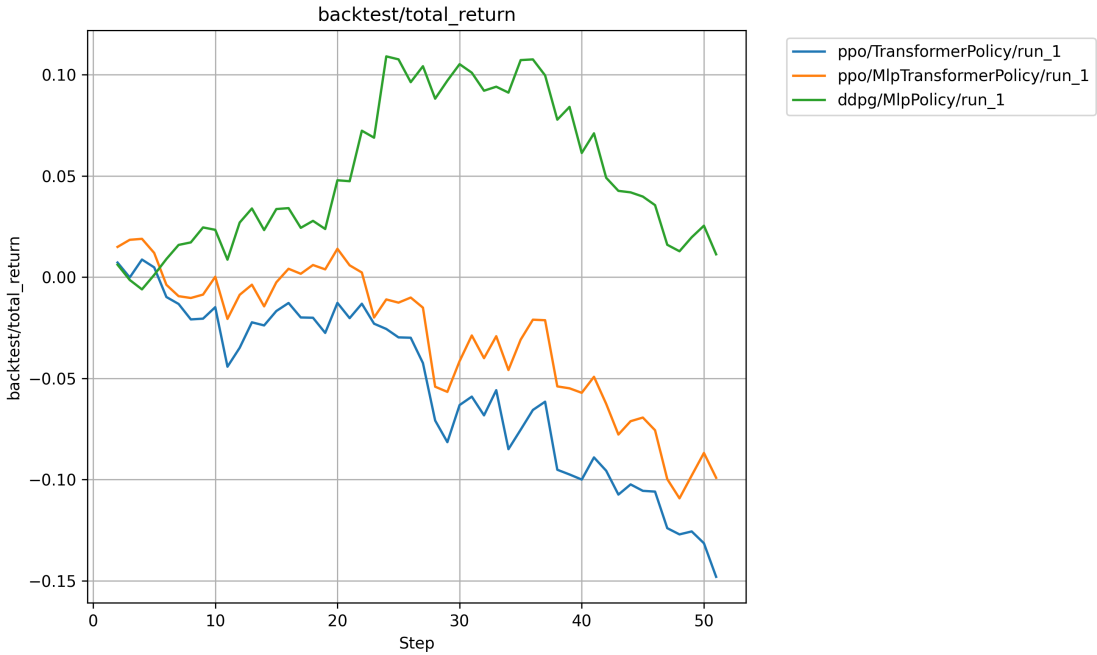


Figure: Out-of-sample total return.

Portfolio Value & Risk-Adjusted Performance

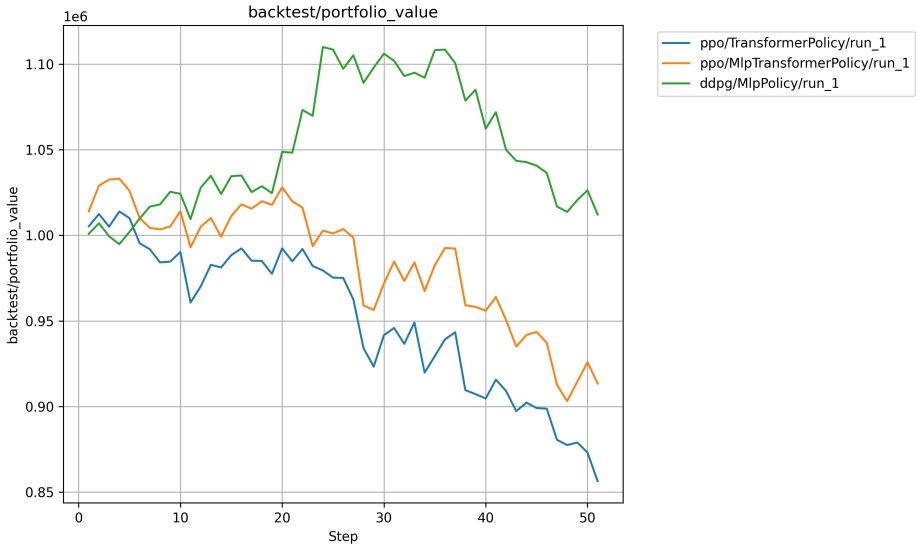


Figure: Portfolio value evolution

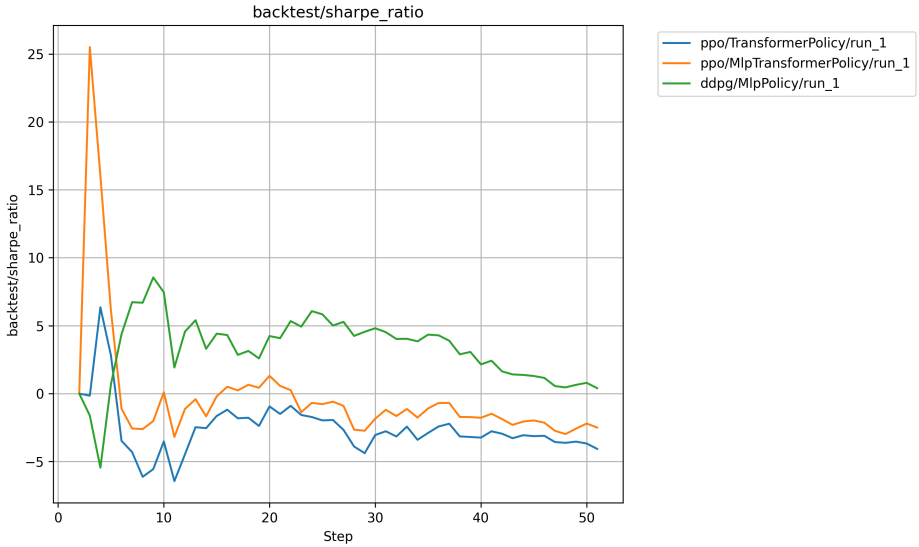


Figure: Sharpe ratio evolution

Key Observation

DDPG (MLP) maintains the strongest portfolio growth and the most consistent risk-adjusted performance during the out-of-sample period, while both Transformer-based PPO variants exhibit gradual degradation.

Fuzzy-Enhanced Risk Management

Motivation

The original environment relied on hard threshold-based controls:

$$a'_t = \min(a_t, 0)$$

which completely blocked buying actions during periods of high market turbulence.

Limitation

- Abrupt action suppression
- Discontinuous behavior
- Difficult learning dynamics

Proposed Solution

A Mamdani Fuzzy Inference System composed of:

1. Fuzzy Action Smoother
2. Fuzzy Reward Shaper

Both components replace hard decisions with smooth, context-dependent risk adjustments.

Fuzzy Action Smoother

Goal

Replace the hard turbulence constraint

$$a'_t = \min(a_t, 0)$$

with a smooth risk-aware action adjustment mechanism.

Inputs

- Raw Action (a_i):

$$\{\text{Sell},\ \text{Hold},\ \text{Buy}\}$$

- Market Turbulence (τ_t):

$$\{\text{Low},\ \text{Medium},\ \text{High}\}$$

Output

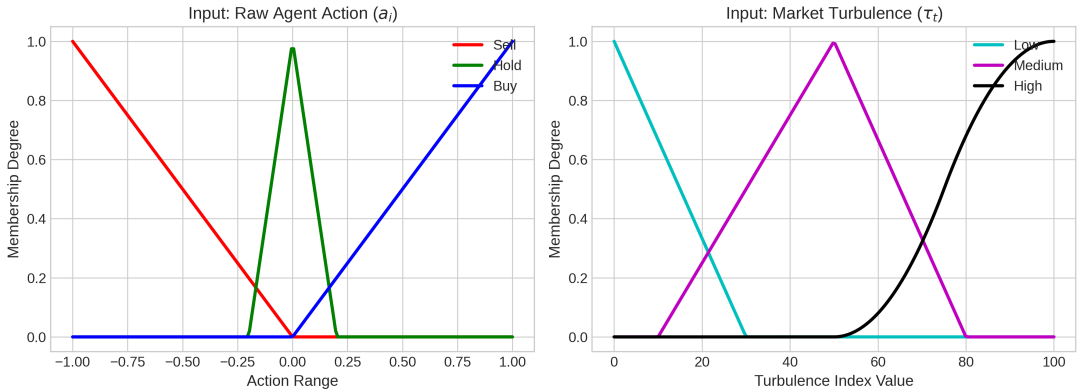
$$a_{t,\text{safe}}$$

Risk-adjusted trading action obtained through Mamdani inference and centroid defuzzification.

Fuzzy Action Smoother Rules

Rule Base

1. **IF** Turbulence is High **THEN** Action is Sell
2. **IF** Turbulence is Medium **AND** Action is Buy **THEN** Action is Hold
3. **IF** Turbulence is Medium **AND** Action is Sell **THEN** Action is Sell
4. **IF** Turbulence is Low **THEN** Action follows the original action



Fuzzy Reward Shaper

Goal

Replace the fixed volatility penalty with a context-dependent fuzzy penalty.

Original reward:

$$r_t = \log\left(\frac{V_{t+1} + \epsilon}{V_t + \epsilon}\right) - \lambda e_t v_t$$

Inputs

- Portfolio Exposure (e_t):

$$\{\text{Low},\ \text{High}\}$$

- VIX Volatility (v_t):

$$\{\text{Calm},\ \text{Stressed},\ \text{Panic}\}$$

Output

$$\phi_t \in [0, 1]$$

Adaptive penalty factor.

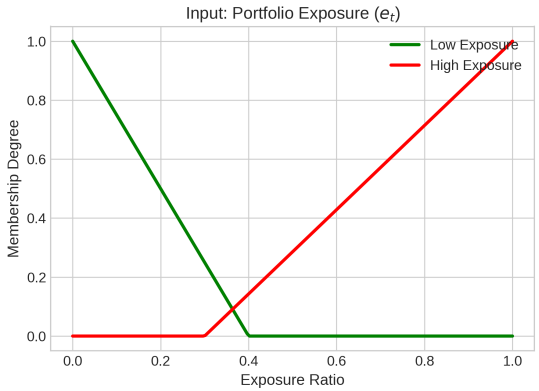
Fuzzy Reward Shaper Rules

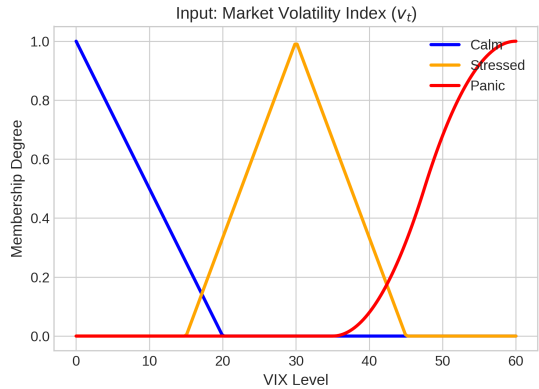
Rule Base

1. **IF** VIX is Panic
   AND Exposure is High
   THEN Penalty is Severe

2. **IF** VIX is Stressed
   AND Exposure is High
   THEN Penalty is Medium

3. **IF** VIX is Calm
   OR Exposure is Low
   THEN Penalty is None





$$r_t = \log\left(\frac{V_{t+1} + \epsilon}{V_t + \epsilon}\right) - \phi_t \lambda_{scale}$$

Genetic Algorithm Hyperparameter Optimization

Motivation

DRL performance is highly sensitive to hyperparameter selection.

Examples include: Learning rate, Batch size, Rollout length etc

Chromosome Representation

Each hyperparameter configuration is encoded as a chromosome:

$$C = [g_1, g_2, \ldots, g_n]$$

where each gene corresponds to a specific hyperparameter.

Fitness Function

$$\mathit{Fitness} = \text{Final Portfolio Value}$$

obtained after training and evaluating the corresponding DRL agent.

Genetic Algorithm Evolution Cycle

Evolutionary Process

1. Generate an initial population of random chromosomes
2. Train and evaluate each DRL agent
3. Compute fitness using the final portfolio value
4. Select the highest-performing individuals
5. Apply crossover to generate offspring
6. Apply mutation to maintain diversity
7. Form the next generation and repeat

Genetic Operators

- **Selection**: favors high-fitness chromosomes
- **Elitism**: preserves the best solution unchanged
- **Uniform Crossover**: combines genes from two parents
- **Mutation**: randomly modifies genes to explore new regions

The cycle is repeated for multiple generations until the best hyperparameter configuration is identified.

Thank You

Questions?

Project Repository

`https://github.com/georrous6/finrl-trading-lab`

Deep Reinforcement Learning Framework for Multi-Asset Trading